Table 2	
Irene Blanco Casallas	William Nimtz
	Sujal Vajire

Table 3	
Max Hortop	Lautaro Garcia
Dashiel Matlock	Lin Tian

Table 4	
Jessica Duran	Ethan Kepros
Mi Zhou	Mohamed Abdulazim Ali Afifi El Habet

Table 5	
Sam Hu	Jack Bajcz
Basma AlMahmood	Samuel Goodluck

Table 1	
Abraham Rai	Alexander Lover
Xin Dong	Jingyi Ge

Table 7	
Saksham Mamtani	Andrew Tran
Sanjida Meherin	Aidana Bekbulatkyzy

Table 6	
Madison Mercer	Md Shariful Islam
Grace Akintan	Michael Dittman

CMSE 801 - Day 9

More modeling with ordinary differential equations, including using `solve_ivp`

Feb 10, 2026

General Course Announcements

- **Reminder**: Homework 1 is due Saturday Feb 21.
    - *Don't wait until the last minute!* -- It shouldn't be overly challenging but it does take time to work through all of the components, which are designed to make sure you review *all* of the fundamentals we've covered up to this point.
- Also, we will have **our first quiz in class on Thursday Feb 12**.
    - This will cover concepts that are covered by Homework 1 (so make sure you're feeling good about your Python fundamentals).
- Homework 2 has been released **today**. It will be due Saturday, **Feb 28**.
    - This will cover using Matplotlib and NumPy as well as a little bit of ODE content (which we're covering this week).

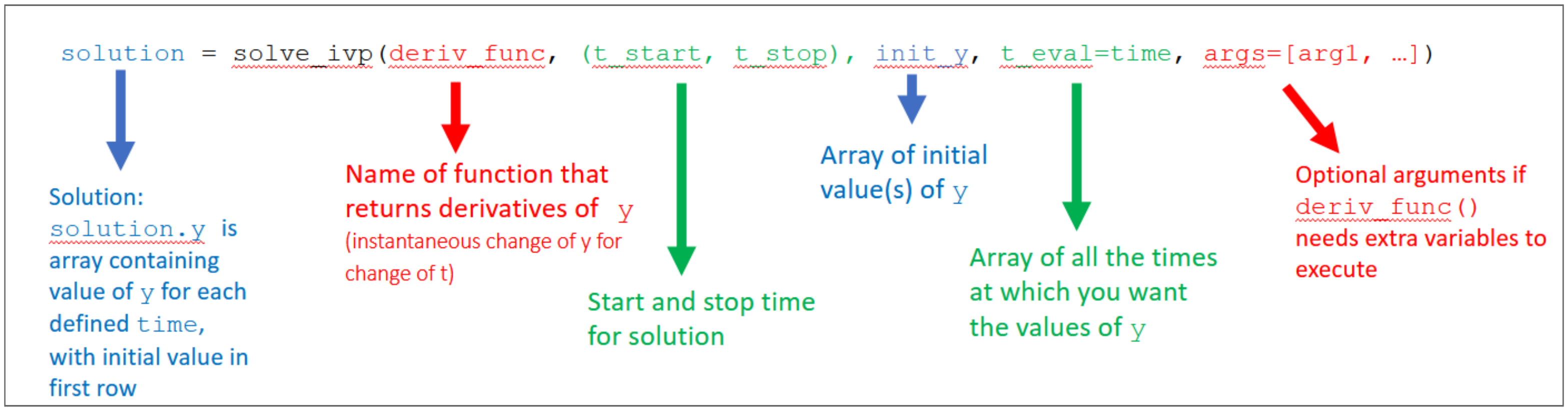
Questions/comments from Day 9 PCA

- What is `solve_ivp` doing?
- One of the most challenging pieces was just figuring out how to use `solve_ivp` for solving an initial value problem
    - format
    - inputs
- coding the air resistance.

`solve_ivp` is finding the numerical solution to the system of ODEs just like you did with the ”update equations”

Let's take a quick look at **the wikipedia page about numerical solving ODEs** and the **`solve_ivp` documentation**.

Let's take a look at what `solve_ivp` needs in order to solve a systems of ODEs



Of course, solve_ivp will expect the function that calculates the derivatives to be in a specific format as well

In [1]:

```
def derivs(time, curr_vals):

    # Declare parameters
    g = -9.81 # m/s^2
    A = 0.4 # m^2
    m = 80.0 # kg

    # Unpack the current values of the variables we wish to "update" from the curr_vals lis
    # Order matters! (needs to match the order of your initial values)
    h, v = curr_vals

    # Right-hand side of odes, which are used to compute the derivative
    dhdt = v
    dvdt = g + (-0.65 * A * v * abs(v))/m

    return dhdt, dvdt # Again, order matters, needs to match order of initial values
```

Or, we could have written this such that the arguments can be passed into the derivatives function

You don't **have** to do this, but it can make the process of exploring the effect of parameters on your model a bit easier. You'll see an example of this in the Day 10 PCA.

In [ ]:

```
def derivs(time, curr_vals, g, A, m):

    # Unpack the current values of the variables we wish to "update" from the curr_vals lis
    # Order matters! (needs to match the order of your initial values)
    h, v = curr_vals

    # Right-hand side of odes, which are used to compute the derivative
    dhdt = v
    dvdt = g + (-0.65 * A * v * abs(v))/m

    return dhdt, dvdt # Again, order matters, needs to match order of initial values
```

Solutions to the problems you were asked to solve in the PCA are in the ICA for today.

Let's take a look at **those** real quick.

Goals for today

- Try to use what you learned in the PCA and what we just dicussed to solve a different, but similar problem.
- Compare your "update equation" solution to the solution provided by `solve_ivp`

Looking forward - Day 10 PCA: Exploring the concept of a ”compartmental model” and how ODEs can be used for these types of model

- You'll look at a "Viral Kinetics" model, which is a compartmental model used to explore how viruses can multiply within your body
- You'll explore how this model can be turned into code and how `solve_ivp` can be used to explore how the model evolves.